Blok B (80 min): praktikum-llm u Docker Composeu, slika, bind mount i imenovani volumen

> 🎯 Ishod vježbe
>
> U osobnom repozitoriju `praktikum-<github-korisnik>` otvoren je **pull request** s grane `feature/compose` prema `main`. PR sadrži:
>
> - **jednostavni** `Dockerfile` (jedna faza) umjesto višefaznog iz tjedna 3,
> - `compose.yaml` sa servisom `llm` (slika `praktikum-llm`) na **host mreži** (`network_mode: host`),
> - `storage.txt` s izlazom koraka 3.7: datoteka iz imenovanog volumena, pročitana iz praznog `alpine` kontejnera,
> - `compose.txt` s izlazom provjera iz koraka 4.
>
> `compose.storage.yaml` iz koraka 3 služi samo za istraživanje i na kraju bloka se briše; u PR-u ga nema.
>
> **Kriteriji prihvaćanja** (svi moraju vrijediti, nastavnik ih provjerava u PR-u):
>
> 1. `Dockerfile` ima jednu fazu (jedan `FROM`), a `docker run --rm praktikum-llm --version` ispisuje verziju iz `version.txt`.
> 2. `docker compose config --quiet` završava bez greške.
> 3. `docker compose run --rm llm status` ispisuje `Ollama is running at http://localhost:11434`.
> 4. `storage.txt` pokazuje datoteku koju ste u koraku 3.7 ručno stvorili u servisu `llm`, pročitanu iz praznog `alpine` kontejnera nakon što je kontejner `llm` obrisan.
> 5. U repozitoriju nema `compose.storage.yaml`, a `llm.py` nije izmijenjen.
> 6. Poruka završnog commita i naslov PR-a slijede Conventional Commits i povećavaju **minor** verziju.
>
> **Predaja:** otvoren PR. Ne spajati ga sami; PR je predaja i ostaje otvoren za pregled.

Sadržaj

0. Grana i priprema

1. Od višefaznog do jednostavnog Dockerfilea

2. Od `docker run` do `compose.yaml`

3. Odakle kontejneru kod i gdje žive podaci: slika, bind mount, imenovani volumen

4. Pospremanje, provjere, commit i pull request

Testove (`Dockerfile.test`, integracijski testovi i servis `tests`) dodajemo u bloku C, kad je i Ollama servis u istom `compose.yaml`.

Gotovo rješenje (stanje repozitorija nakon koraka 4) nalazi se u docker-lab/blok-b/. Ako je vaš repozitorij nakon tjedna 3 već Python paket (`src/praktikum_llm/`, `pyproject.toml`), razlike su u okvirima "Paket" i u docker-lab/paket/.

0. Grana i priprema

```
cd ~/praktikum-$(gh api user -q .login)
git checkout main && git pull
git checkout -b feature/compose
docker compose version            # Docker Compose version v2 or newer
curl -s http://localhost:11434    # Ollama is running (week 2)
ollama list                       # llama3.2:1b must be listed; otherwise:
ollama pull llama3.2:1b
docker pull alpine:3.24           # used in step 3.7
```

Polazište je stanje nakon PR-a iz tjedna 3: višefazni `Dockerfile` (faze `base`, `test`, `runtime`), `.dockerignore`, `requirements.txt`, `requirements-dev.txt`, `tests/`, `version.txt`.

Docker Desktop (Windows, macOS): uključiti host mrežu. *Settings → Resources → Network → Enable host networking* (Docker Desktop 4.34 ili noviji), zatim *Apply & restart*. Bez toga `network_mode: host` na Desktopu ne vidi servise na vašem računalu. Na Docker Engineu u Linuxu ili WSL-u host mreža radi bez postavki.

1. Od višefaznog do jednostavnog Dockerfilea

Višefazni Dockerfile iz tjedna 3 radi tri stvari odjednom (zajednička baza, testovi u buildu, ne-root runtime). Ovaj tjedan pratimo što se događa s kodom između IDE-a, slike, kontejnera i volumena, pa krećemo od najjednostavnijeg recepta, istog kao na predavanju. Testove više ne pokreće build; u bloku C dobit će vlastiti kontejner.

Cijeli `Dockerfile` zamijeniti s:

```
# Simple, single-stage image of the llm CLI (week 4 replaces the multi-
stage build from week 3).
#   docker build -t praktikum-llm .
#   docker run --rm praktikum-llm --version
FROM python:3.13-alpine

# PYTHONDONTWRITEBYTECODE: do not write .pyc files; PYTHONUNBUFFERED: logs
appear immediately.
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

# Dependencies first: this layer is rebuilt only when requirements.txt
changes.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Source code last: it changes most often.
COPY version.txt ollama_client.py llm.py ./

ENTRYPOINT ["python", "llm.py"]
CMD ["--help"]
```

```
docker build -t praktikum-llm .
docker run --rm praktikum-llm --version       # llm x.y.z from version.txt
docker run --rm --entrypoint id praktikum-llm # uid=0(root): no USER line,
the container runs as root
docker history praktikum-llm                  # one layer per instruction,
no hidden stages
```

	Višefazni (tjedan 3)	**Jednostavni (tjedan 4)**
Faze	`base`, `test`, `runtime`	jedna
Testovi	`RUN pytest` za vrijeme builda (`--target test`)	u kontejneru iz `Dockerfile.test` (blok C)
Korisnik	`app` (uid 10001)	root
Kada ga koristiti	produkcijska slika	učenje, razvoj, jasan pregled slojeva

Ne-root korisnik iz tjedna 3 je i dalje dobra praksa za produkciju; vraćamo ga kad sliku budemo objavljivali iz CI-ja.

> **Paket.** Ako je alat paket iz bloka C tjedna 3, Dockerfile kopira `pyproject.toml`, `version.txt` i `src/`, pa radi `pip install --no-cache-dir -e .` (*editable*: naredba `llm` izvodi kod iz `/app/src`) i ima `ENTRYPOINT ["llm"]`; gotova datoteka.

2. Od `docker run` do `compose.yaml`

Do sada smo alat pokretali ovako:

```
docker build -t praktikum-llm .
docker run --rm --network host -e OLLAMA_BASE_URL=http://localhost:11434
praktikum-llm status
```

Svaka zastavica (`--network`, `-e`, `-v`, `-p`, ime slike) je konfiguracija koju treba zapamtiti i prepisati bez greške. Docker Compose istu konfiguraciju zapisuje u datoteku `compose.yaml`, koja ide u git zajedno s kodom.

U korijen repozitorija dodati `compose.yaml`:

```
# Week 4, block B: the CLI as a Compose service.
# It uses the HOST network: inside the container localhost:11434 is the
Ollama
# that runs on your machine (from week 2), exactly like `docker run --
network host`.
name: praktikum

services:
  llm:
    build:
      context: .
    image: praktikum-llm
    network_mode: host
    environment:
      OLLAMA_BASE_URL: http://localhost:11434
      OLLAMA_MODEL: ${OLLAMA_MODEL:-llama3.2:1b}
    profiles: [cli]              # not started by `docker compose up`, only
by `docker compose run llm ...`
```

Ključ	Značenje	Ekvivalent u `docker` naredbama
`name`	ime projekta; prefiks svih kontejnera, mreža i volumena	`docker compose -p praktikum`
`build.context`	odakle se gradi slika (direktorij s `Dockerfile`)	`docker build .`
`image`	ime (tag) slike koja nastaje ili se preuzima	`-t praktikum-llm`
`network_mode: host`	kontejner dijeli mrežu s hostom	`--network host`
`environment`	varijable okoline u kontejneru	`-e KEY=value`
`${OLLAMA_MODEL:-llama3.2:1b}`	vrijednost iz okoline ili iz `.env` pokraj `compose.yaml`, inače zadana	(ljuska)
`profiles`	servis se ne pokreće s `docker compose up`, nego samo kad ga se izravno imenuje	–

```
docker compose config                          # the resolved file:
variables filled in, defaults added
docker compose run --rm --build llm status     # build the image, start ONE
container, remove it afterwards
docker compose run --rm llm models
docker compose run --rm llm ask "What is Docker Compose in one sentence?"
docker compose run --rm llm --version
```

`docker compose run` pokreće jedan kontejner servisa s naredbom iza imena servisa (umjesto `CMD`) i, uz `--rm`, briše ga na izlasku. To je **kratkoživući** kontejner: obavi jedan posao i završi. Kontejner koji radi dok

ga ne zaustavimo (poslužitelj) zovemo **dugoživući**; takav ćemo dodati u bloku C.

3. Odakle kontejneru kod i gdje žive podaci: slika, bind mount, imenovani volumen

Na predavanju smo vidjeli da je kod u slici zamrznut (SHA-256). Sada to provjeravamo sami, kroz Compose, na servisu koji samo "spava" da bismo mogli gledati u njega. U korijen repozitorija dodati `compose.storage.yaml`:

```
# Week 4, block B, step 3: where does the code inside a container come
from, and where does data live?
# Run every command with: docker compose -f compose.storage.yaml ...
name: praktikum-storage

services:
  llm:
    build:
      context: .
    image: praktikum-llm
    entrypoint: ["sleep", "infinity"]   # keep the container running so we
can look inside it
```

Da ne pišemo `-f` u svakoj naredbi, koristimo kraticu:

```
alias dcs='docker compose -f compose.storage.yaml'
```

3.1 Build. Izgraditi sliku i pokrenuti kontejner:

```
dcs up -d --build
dcs exec llm ls -l /app                                    # the files COPY
put into the image
docker image inspect praktikum-llm --format '{{.Id}}'      # image ID =
SHA-256 of the image
sha256sum llm.py; dcs exec llm sha256sum llm.py            # same hash:
same file
```

3.2 Izmjena u IDE-u, kod iz slike. U IDE-u (VS Code) u `llm.py` promijeniti opis parsera u `"Talk to a local Ollama model (edited in IDE)."` i spremiti.

```
dcs exec llm grep -c "edited in IDE" llm.py     # 0: the container still
has the old file
sha256sum llm.py; dcs exec llm sha256sum llm.py # the hashes differ now
dcs up -d                                       # nothing changes: the
image did not change
dcs up -d --build                               # new build -> new image ID
```

```
-> new container
dcs exec llm grep -c "edited in IDE" llm.py     # 1
```

Zaključak: kontejner vidi kod iz trenutka builda. Izmjena u IDE-u ulazi u kontejner samo kroz novi build.

3.3 Bind mount. Direktorij projekta montirati preko radnog direktorija kontejnera (`WORKDIR /app`). U servis `llm` dodati:

```
    volumes:
      - ./:/app                          # bind mount: the project folder on
the host
```

```
dcs up -d                                # Compose recreates the
container because the config changed
dcs exec llm ls -la /app                 # the whole project, including
.git and .env: .dockerignore does not apply to mounts
```

3.4 Izmjena u IDE-u, bind mount. U `llm.py` promijeniti opis u `"Talk to a local Ollama model (edited in IDE, round 2)."` i spremiti.

```
dcs exec llm grep -c "round 2" llm.py    # 1, without a build: the
container reads the host folder
```

Bind mount zaobilazi sliku: tako se radi u razvoju (spremiš u IDE-u, kontejner odmah vidi). U produkciji nema bind mounta, ondje vrijedi samo ono što je u slici.

3.5 Imenovani volumen. Bind mount zamijeniti imenovanim volumenom:

```
    volumes:
      - llm-code:/app                    # named volume over the working
directory
```

i na kraj datoteke dodati:

```
volumes:
  llm-code:
```

```
dcs up -d
docker volume ls                         # praktikum-storage_llm-code
dcs exec llm ls -l /app                  # not empty: Docker copied /app
```

```
from the IMAGE into the new, empty volume
dcs exec llm grep -c "round 2" llm.py      # 0: the image was built in 3.2,
before round 2
```

3.6 Izmjena u IDE-u, imenovani volumen. U `llm.py` opis promijeniti u `"... (edited in IDE, round 3)."`.

```
dcs exec llm grep -c "round 3" llm.py      # 0: the volume is not connected
to the project folder
dcs up -d --build                          # even a new image does not
help...
dcs exec llm grep -c "round 3" llm.py      # ...0: Docker fills a volume
from the image only while it is EMPTY
docker run --rm --entrypoint grep praktikum-llm -c "round 3" llm.py   # 1:
the new image has it, the volume does not
```

Imenovani volumen preko koda je zamka: jednom napunjen, volumen "zamrzne" staru verziju i nadjača novu sliku. Kod pripada slici; volumen je za **podatke** koje aplikacija stvara. To radimo u sljedećem koraku.

3.7 Imenovani volumen za podatke i prazni alpine kontejner. Servisu `llm` dodati drugi imenovani volumen, `llm-data`, na `/data`:

```
    volumes:
      - llm-code:/app
      - llm-data:/data                 # named volume for data the
application creates
```

a pod `volumes:` na kraju datoteke dodati `llm-data:`.

```
dcs up -d
dcs exec llm ls -la /data                  # empty: /data does not exist in
the image, so there is nothing to copy
```

U kontejneru `llm` ručno napraviti direktorij i datoteku:

```
dcs exec llm sh
  mkdir -p /data/notes
  echo "created by $(hostname) at $(date -Iseconds)" >
/data/notes/hello.txt
  ls -la /data/notes
  exit
```

Isti volumen sada montiramo u **novi, prazni** `alpine` kontejner, koji nema ni Pythona ni našeg koda. U `compose.storage.yaml` dodati servis:

```
  data-inspect:
    # a new, empty alpine container that only mounts the llm-data volume
    image: alpine:3.24
    volumes:
      - llm-data:/data:ro               # read-only: inspecting must not
change the data
    command: ["sleep", "infinity"]
```

```
dcs up -d data-inspect && dcs exec -it data-inspect ls -la /data/notes    #
the file created in llm, read by alpine
docker volume inspect praktikum-storage_llm-data          # Driver,
Mountpoint, CreatedAt, Compose labels
dcs down                                                  # remove the llm
container...
dcs up -d data-inspect && dcs exec -it data-inspect sh    # ...the data is
still there
dcs exec data-inspect cat /data/notes/hello.txt | tee storage.txt   #
evidence for criterion 4
```

`alpine` vidi datoteku iako je nastala u drugom kontejneru, a ostaje i nakon što kontejner `llm` više ne postoji: podaci su u volumenu, ne u kontejneru. `docker volume inspect` pokazuje da volumenom upravlja Docker (`Driver: local`, `Mountpoint` u `/var/lib/docker/volumes/...`, na Docker Desktopu unutar njegovog VM-a), a ne neka putanja na vašem disku.

Zašto imenovani volumeni u produkciji.

Prednost	Što znači
Životni vijek neovisan o kontejneru	nova verzija slike = novi kontejner, a baza, modeli i datoteke korisnika ostaju (`down` ih ne briše)
Ne ovisi o putanji na hostu	`compose.yaml` radi isto na svakom poslužitelju; nema `/home/<netko>/...` ni razlika Windows/Linux
Docker upravlja vlasništvom i inicijalizacijom	prazni volumen dobiva sadržaj i vlasnika iz slike; nema ručnog `chown` na hostu
Dijeljenje među kontejnerima	aplikacija piše, drugi kontejner (backup, analiza) čita, i to samo za čitanje (`:ro`)
Backup i migracija	pomoćni kontejner (`alpine` + `tar`) zapakira volumen bez zaustavljanja ostatka sustava
Upravljački programi (*volume drivers*)	isti `compose.yaml`, a podaci na NFS-u ili mrežnom disku u cloudu (npr. GCP Persistent Disk, tema 8)

Prednost	Što znači
Performanse na Docker Desktopu	volumen je u Dockerovom VM-u, pa je brži od bind mounta koji ide kroz dijeljenje datoteka s macOS-om ili Windowsima
Nema slučajnih izmjena s hosta	nitko ne može "samo popraviti" datoteku u editoru na poslužitelju

Pravilo: **kod u slici, podaci u imenovanom volumenu, bind mount samo u razvoju.**

Vratiti `llm.py`. Izmjene opisa iz koraka 3.2–3.6 ne idu u PR:

```
git checkout llm.py
```

U `.dockerignore` dodati `storage.txt` (izlaz provjere ne ide u sliku). Projekt `praktikum-storage` i `compose.storage.yaml` brišemo na kraju bloka (korak 4).

> **Paket.** S *editable* instalacijom (korak 1) kod koji se izvodi je u `/app/src/praktikum_llm/`. U koracima 3.2–3.6 umjesto `llm.py` pratiti `src/praktikum_llm/cli.py` (npr. `dcs exec llm grep -c "edited in IDE" src/praktikum_llm/cli.py`), a `dcs exec llm llm --help` pokazuje koji opis alat stvarno ispisuje. Korak 3.7 je isti.

4. Pospremanje, provjere, commit i pull request

Obrisati `compose.storage.yaml`. Datoteka je bila alat za istraživanje (kontejner koji samo spava, volumen preko koda, što je namjerna zamka iz 3.6) i ne pripada repozitoriju. Prvo obrisati njezine kontejnere i volumene, dok Compose još zna koji su, a zatim i samu datoteku:

```
docker compose -f compose.storage.yaml down -v    # containers, network and
the volumes llm-code, llm-data
docker volume ls                                  # no praktikum-storage_*
volumes left
rm compose.storage.yaml                           # already committed?
then: git rm compose.storage.yaml
unalias dcs
```

Redoslijed je bitan: nakon `rm` Compose više ne zna za projekt `praktikum-storage`, pa bi volumene trebalo brisati ručno (`docker volume rm praktikum-storage_llm-code praktikum-storage_llm-data`).

Provjere. Izlaz provjera ide u `compose.txt`:

```
{
  echo "## dockerfile";  grep -c "^FROM" Dockerfile; docker run --rm
praktikum-llm --version
  echo "## config";      docker compose config --quiet && echo
"compose.yaml is valid"
```

```
  echo "## llm status";  docker compose run --rm --build llm status 2>&1 |
tail -1
  echo "## storage";     ls compose.storage.yaml 2>&1; git diff --stat
llm.py
} | tee compose.txt
```

U `.dockerignore` dodati `compose.txt`.

Poruka završnog commita. Završni commit mora prema specifikaciji Conventional Commits povećati **minor** verziju (`x.Y.z`). Koji tip (prefiks) poruke to postiže, znate iz tjedna 3; u naredbama ispod umjesto njega piše `<type>`. Isti prefiks koristite i u naslovu PR-a.

```
git add Dockerfile compose.yaml storage.txt .dockerignore compose.txt
git status                        # .env, .venv and compose.storage.yaml
must NOT be listed; llm.py must NOT be modified
git commit -m "<type>: run the CLI with Docker Compose"
git push -u origin HEAD
gh pr create --base main --title "<type>: run the CLI with Docker Compose"
--body "Week 4, block B. Checks: see compose.txt and storage.txt"
```

Predaja (praktični zadatak 4, dio B): otvoren PR iz `feature/compose` u `main` repozitorija `praktikum-<github-korisnik>`, s datotekama iz ishoda te `compose.txt` i `storage.txt` koji potvrđuju kriterije 1–5 (kriterij 6 provjerava se iz poruke commita i naslova PR-a). PR ne spajati sami.

Česti problemi

Simptom	Rješenje
`Ollama is NOT running at http://localhost:11434` iz `docker compose run --rm llm status`	Docker Desktop: uključiti *Enable host networking* (korak 0). Ako ni tada ne radi (npr. Ollama u WSL-u, Docker Desktop u Windowsima), privremeno zamijeniti `network_mode: host` s `extra_hosts: ["host.docker.internal:host-gateway"]` i `OLLAMA_BASE_URL: http://host.docker.internal:11434`. U bloku C Ollama ionako prelazi u Compose.
`No services to build`	Servis `llm` je u profilu `cli`: `docker compose --profile cli build` ili `docker compose run --build llm ...`
`validating compose.yaml: services.build ...` ili slična greška sheme	Uvlaka u YAML-u: `build:` i `volumes:` moraju biti uvučeni ispod imena servisa (2 razmaka više), a stavke `- ...` ispod `volumes:`
`service ... declares mutually exclusive network_mode and networks`	Na host mreži nema `networks:`; ni `ports:` nema smisla (Compose ga tiho ignorira), jer kontejner koristi portove hosta izravno
U 3.6 i nakon `dcs up -d --build` kontejner vidi stari `llm.py`	Očekivano: neprazan imenovani volumen nadjačava sadržaj slike. Novi sadržaj iz slike dobiva se tek nakon `dcs down -v`

Simptom	Rješenje
	(briše volumen)
`cat: can't open '/data/notes/hello.txt'` u `data-inspect`	Datoteka nije stvorena u `/data/notes/` kontejnera `llm`, ili je `llm-data` montiran na drugu putanju; provjeriti `dcs exec llm ls -laR /data`
`dcs: command not found`	Alias vrijedi samo u terminalu u kojem je definiran; ponoviti `alias dcs=...` ili pisati puni `docker compose -f compose.storage.yaml`
`compose.storage.yaml: no such file` kod `down -v`, a volumeni `praktikum-storage_*` su ostali	Datoteka je obrisana prije `down -v`: `docker volume rm praktikum-storage_llm-code praktikum-storage_llm-data`
`llm.py` je u `git status` kao izmijenjen	Izmjene iz koraka 3 ne idu u PR: `git checkout llm.py`